



Waipapa
Taumata Rau
**University
of Auckland**

Classification and Regression

SOFTENG 753: Machine Learning Techniques
and Applications

>>> Semester 2, 2026

>>> Maryam Hemmati
m.hemmati@auckland.ac.nz

Content



- **Multiclass Classification**
 - **Classifying Newswires**
- **Regression Terminology**
- **Regression Example**
 - **Predicting House Prices**

➤ Multiclass Classification

- We have **multiclass classification** when we have **more than two classes**.
 - In the binary classification example, we classified vector inputs into **two mutually exclusive** classes.
- We will learn to build a model to classify **Reuters newswires** into **46 mutually exclusive topics**.
- Each data point should be classified into **only one category**, the problem is more specifically an instance of **single-label, multiclass classification**.
- We will have a **multilabel, multiclass classification** problem if each data point could belong to **multiple categories**.

➤ Multiclass Classification

- We will use **Reuters dataset**
 - A set of **short newswires** and their **topics**, published by Reuters in 1986.
 - There are **46 different topics**.
 - Some topics are more represented than others.
 - Each topic has **at least 10 examples** in the training set.
 - There are **8,982 training** examples and **2,246 test** examples.

➤ Classifying Newswires

- The **Reuters dataset** comes [packaged](#) with [Keras](#).
 - Like the MNIST and IMDB dataset.
- Each example is a [list of integers](#) (word indices).
 - Each [integer](#) stands for a [specific word in a dictionary](#).
 - Similar to IMDB reviews.
- The label associated with an example is an [integer](#) between 0 and 45.
 - This is a [topic index](#).

```
from keras.datasets import reuters

(train_data, train_labels), (test_data, test_labels) = reuters.load_data(
    num_words=10000
)
```

Loading [Reuters](#) dataset.

➤ Classifying Newswires – Data Preparation

- The argument [num_words=10000](#) restricts the data to the [10,000 most frequently occurring](#) words found in the data.
 - Similar to the IMDB dataset.
- We can vectorize the data with the exact same code as in the previous example with IMDB dataset.
 - We use [multi-hot encoding](#).

```
# Vectorized training data
x_train = multi_hot_encode(train_data, num_classes=10000)
# Vectorized test data
x_test = multi_hot_encode(test_data, num_classes=10000)
```

Encoding the [input](#) data

➤ Classifying Newswires – Data Preparation

- To vectorize the labels, there are two possibilities
 - We can leave the labels untouched as **integers**.
 - We can use **one-hot encoding**.
- **One-hot encoding** is a widely used format for **categorical data**, also called **categorical encoding**.
- **One-hot encoding** of the labels in our example
 - Embedding each label as an **all-zero vector** with a **1** in the place of the **label index**.

➤ Classifying Newswires – Data Preparation

- We can write a simple function for **one-hot encoding**.
- There is also a **built-in** way to do this in **Keras**.

```
def one_hot_encode(labels, num_classes=46):  
    results = np.zeros((len(labels), num_classes))  
    for i, label in enumerate(labels):  
        results[i, label] = 1.0  
    return results
```

```
# Vectorized training labels  
y_train = one_hot_encode(train_labels)  
# Vectorized test labels  
y_test = one_hot_encode(test_labels)
```

```
from keras.utils import to_categorical  
  
y_train = to_categorical(train_labels)  
y_test = to_categorical(test_labels)
```

Classifying Newswires

Let's have a look and try the example in [Jupyter notebook](#)

➤ Classifying Newswires – Building Model

- Compared to binary classification, we have a **new constraint** here.
 - The number of **output classes** has gone from 2 to 46.
- Each layer in a stack of Dense layers can **only access** information present in the output of the **previous layer**.
- If one layer **drops** some information relevant to the classification problem, this information can **never be recovered** by later layers.
 - Each layer can potentially become an **information bottleneck**.
 - They can **permanently drop** relevant information.

```
model = keras.Sequential(  
    [  
        layers.Dense(64, activation="relu"),  
        layers.Dense(64, activation="relu"),  
        layers.Dense(46, activation="softmax"),  
    ]  
)
```

Model definition.

➤ Classifying Newswires – Building Model

- We **end** the model with a Dense layer of **size 46**.
- For each input sample, the network will output a **46-dimensional vector**.
 - Each entry in this vector (each dimension) will encode a different **output class**.
- The last layer uses a **softmax activation**.
 - The same activation we used in the MNIST example.
 - The model will output a **probability distribution** over the **46** different output classes.
- For every input sample, the model will produce a **46-dimensional output vector**.
 - The **output[i]** is the **probability** that the sample belongs to **class i**.
 - The 46 scores will **sum to 1**.

➤ Classifying Newswires – Compiling Model

- The best loss function to use in this case is **categorical_crossentropy**.
- **Categorical_crossentropy** measures the **distance** between the **probability distribution** outputted by the model and the true distribution of the labels.
- By **minimizing the distance** between these two distributions, we train the model to output something **as close as possible** to the true labels.

➤ Classifying Newswires – Compiling Model

- We monitor **top-k accuracy**.
 - It measures whether the correct class was **among the top-k predictions** of the model.
 - We use **top-3 accuracy** for our model.
- Accuracy is a bit of a crude metric in this case.
 - If the model has the correct class as its **second choice** for a given sample, with an incorrect first choice, the model will still have an **accuracy of zero** on that sample.
- We use **adam** (Adaptive Moment Estimation) optimizer.

```
top_3_accuracy = keras.metrics.TopKCategoricalAccuracy(  
    k=3, name="top_3_accuracy"  
)  
model.compile(  
    optimizer="adam",  
    loss="categorical_crossentropy",  
    metrics=["accuracy", top_3_accuracy],  
)
```

Compiling the model.

➤ Classifying Newswires– Validating and Training

- We set apart **1,000 samples** in the training data to use as a **validation set**.
- We train the model
 - **20 epochs**
 - 20 **iterations** over **all samples** in the training data.
 - **Mini-batches** of **512 samples**.

```
x_val = x_train[:1000]  
partial_x_train = x_train[1000:]  
y_val = y_train[:1000]  
partial_y_train = y_train[1000:]
```

Setting aside a **validation set**.

```
history = model.fit(  
    partial_x_train,  
    partial_y_train,  
    epochs=20,  
    batch_size=512,  
    validation_data=(x_val, y_val),  
)
```

Training the model.

➤ Classifying Newswires – Evaluating Model

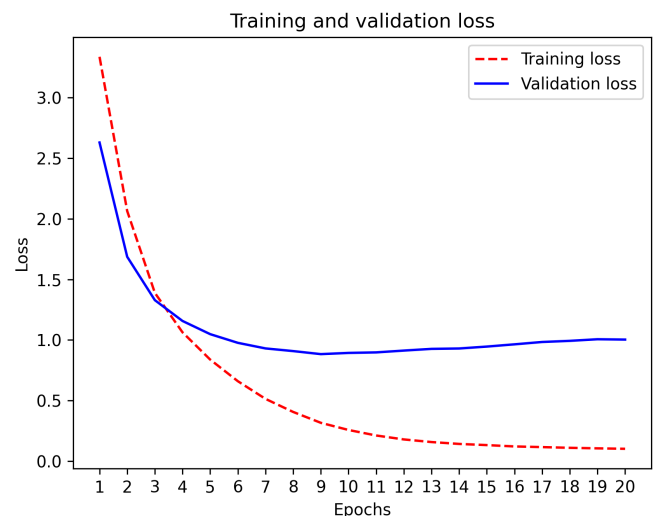
- We evaluate our model by checking its loss and accuracy curves using [Matplotlib](#).

```
loss = history.history["loss"]
val_loss = history.history["val_loss"]
epochs = range(1, len(loss) + 1)
plt.plot(epochs, loss, "r--", label="Training loss")
plt.plot(epochs, val_loss, "b", label="Validation loss")
plt.title("Training and validation loss")
plt.xlabel("Epochs")
plt.xticks(epochs)
plt.ylabel("Loss")
plt.legend()
plt.show()
```

Plotting the [training](#) and [validation](#) loss.

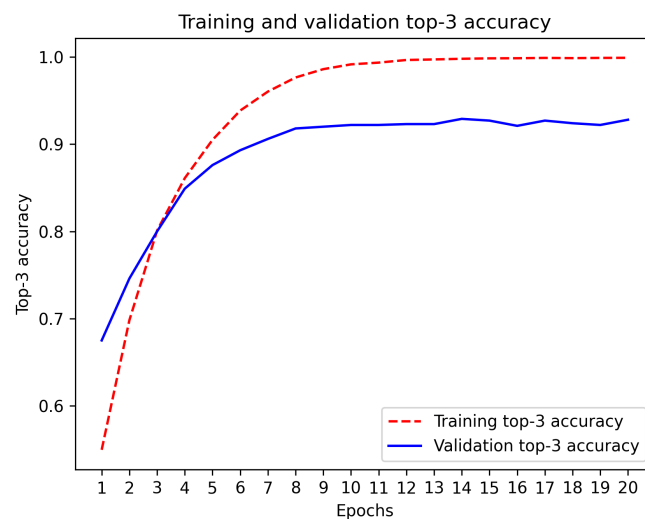
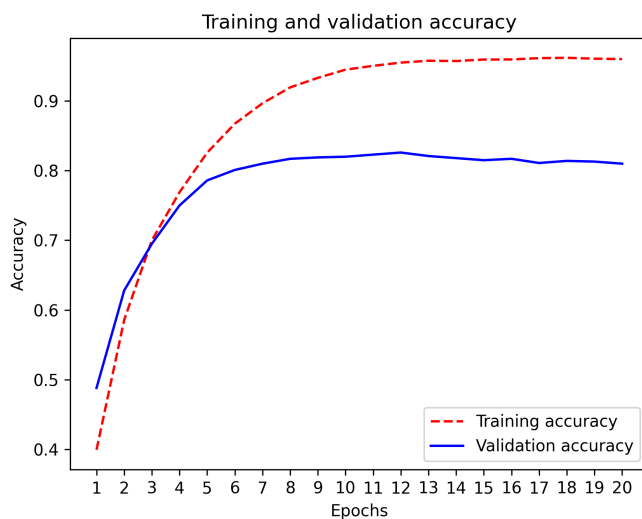
➤ Classifying Newswires – Evaluating Model

- The training loss [decreases](#) with every epoch.
- The [validation](#) loss seems to [peak](#) at the [ninth epoch](#).
 - The model begins to [overfit](#) after nine epochs.
- We can train a new model from [scratch](#) for [nine epochs](#) and then evaluate it on the test set.



The [training](#) and [validation](#) loss.

➤ Classifying Newswires – Evaluating Model



The **top-3 accuracy** is a more reasonable metric in multiclass classification.

➤ Classifying Newswires – Evaluating Model

- Training our model for **nine epochs** results in an accuracy of approximately **80%**.
- With a balanced binary classification problem, the accuracy reached by a purely **random classifier** would be **50%**.
- In this case, we have **46 classes**, and they may not be **equally represented**.
- What could check what would be the accuracy of a random baseline?

```
>>> import copy
>>> test_labels_copy = copy.copy(test_labels)
>>> np.random.shuffle(test_labels_copy)
>>> hits_array = np.array(test_labels == test_labels_copy)
>>> hits_array.mean()
0.18655387355298308
```

- A random classifier would score around **19%** classification accuracy.
 - The results of our model seem pretty good.

➤ Classifying Newswires – Predictions

- Calling the model's predict method on new samples returns a **class probability distribution** over all **46 topics** for each sample.

```
predictions = model.predict(x_test)
```

- Each entry in "predictions" is a vector of length **46**.
- The **coefficients** in this vector **sum to 1**, as they **form a probability distribution**.
- The **largest** entry is the **predicted class**
 - This is the class with the **highest probability**.

➤ Classifying Newswires – Other Approach

- Another way to **encode** the labels would be to leave them **untouched as integer** tensors.
- The only thing this approach would change is the **choice** of the **loss function**.
 - With integer labels, we should use **sparse_categorical_crossentropy**.

```
model.compile(  
    optimizer="adam",  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"],  
)
```

- This new loss function is still **mathematically** the same as categorical_crossentropy.
 - It just has a different **interface**.

➤ Classifying Newswires – Further Experiments

- We introduce an **information bottleneck** by having intermediate layers that are significantly less than 46-dimensional
- We make the **second layer** to a **4-dimensional** layer.
 - The model now peaks at approximately **71% validation accuracy**, an **8% absolute drop**.
- The model is able to **cram most** of the necessary information into these 4-dimensional representations, but **not all** of it.

➤ You try

- Using **larger** or **smaller** layers: 32 units, 128 units, and so on.
- Using a **single intermediate** layer, or **three intermediate** layers.

➤ Regression Terminology

➤ Scalar regression

- A task where the target is a **continuous scalar value**.
- Predicting house prices is a good example: the different target prices form a **continuous space**.

➤ Vector regression

- A task where the target is a **set of continuous values**: for example, a continuous vector.
- We do vector regression when we do regression against **multiple values** (such as the coordinates of a bounding box in an image).

➤ Predicting House Prices

- The goal in classification problems is to **predict a single discrete label** of an input data point.
- **Regression** is another common type of machine learning problem that aims to predict a **continuous value** instead of a discrete label.
 - Predicting the time that a software project will take to complete given its specifications.
- We want to predict the **median price of homes** in different areas of California, based on data from the **1990 census**.
- Each data point in the dataset represents information about a **"block group"**
 - A group of homes located in the same area.
 - Think of it as a **district**.

➤ Predicting House Prices – Data Preparation

- For each district, we know
 - The **longitude** and **latitude** of the approximate geographic center of the area.
 - The **median age** of houses in the district.
 - The **population** of the district.
 - The districts are pretty small. the average population is 1,425.5.
 - The **total number** of households.
 - The **median income** of those households.
 - The **total number of rooms** in the district, across all homes located there.
 - This is typically in the low thousands.
 - The **total number of bedrooms** in the district.
- The goal is to use these **eight variables** to predict the **median value of the houses** in the district.

➤ Predicting House Prices – Data Preparation

➤ The California Housing Price dataset has two versions:

- The **"small"** version with just 600 districts,
- The **"large"** version with 20,640 districts.

➤ We use the small version, because real-world datasets can often be tiny.

- We need to know how to handle such cases.

➤ We have 480 training samples and 120 test samples.

- Each with 8 numerical features.

➤ The prices are between \$60,000 and \$500,000.

```
from keras.datasets import california_housing

# Make sure to pass version="small" to get the right dataset.
(train_data, train_targets), (test_data, test_targets) = (
    california_housing.load_data(version="small")
)
```

Loading the California Housing Prices dataset.

➤ Predicting House Prices – Data Preparation

➤ Feeding values with wildly different ranges into a neural network will be problematic.

- The model might be able to automatically adapt to such heterogeneous data.
- It would definitely make learning more difficult.

➤ A widespread best practice to deal with such data is to do feature-wise normalization.

- For each feature in the input data (a column in the input data matrix), we subtract the mean of the feature and divide by the standard deviation.
- The feature is centered around 0 and has a unit standard deviation.

```
mean = train_data.mean(axis=0)
std = train_data.std(axis=0)
x_train = (train_data - mean) / std
x_test = (test_data - mean) / std
```

Normalizing the data.

The quantities used for normalizing the test data are computed using the training data.

We should never use in any quantity computed on the test data.

➤ Predicting House Prices – Data Preparation

- We should also **scale** the **targets**.
 - Our normalized inputs have their value in a small range close to 0.
 - Our model's weights are initialized with small random values.
 - Our model's prediction will also be small values when we start training.
- If the targets are in the range 60,000–500,000, the model is going to need **very large weight values** to output those.
 - With a small learning rate, it would take a very long time to get there.
- The simplest fix is to **divide all target** values by **100,000**.
 - The smallest target becomes **0.6**, and the largest becomes **5**.
 - We can then convert the model's predictions back to dollar values by **multiplying** them by **100,000** accordingly.

```
y_train = train_targets / 100000
y_test = test_targets / 100000
```

Scaling the targets.

➤ Predicting House Prices – Building Model

- We will use a very small model with **two intermediate layers**, each with **64 units**.
 - Because very few samples are available.
- The **less training data** we have, the **worse overfitting** will be.
 - Using a **small model** is one way to **mitigate overfitting**.
- The model ends with a single unit and **no activation**.
 - It will be a **linear layer**.
 - This is a typical setup for **scalar regression** where we are trying to predict a **single continuous value**.

```
def get_model():
    # Because you need to instantiate the same model multiple times,
    # you use a function to construct it.
    model = keras.Sequential(
        [
            layers.Dense(64, activation="relu"),
            layers.Dense(64, activation="relu"),
            layers.Dense(1),
        ]
    )
    model.compile(
        optimizer="adam",
        loss="mean_squared_error",
        metrics=["mean_absolute_error"],
    )
    return model
```

Model definition.

The last layer is **purely linear**, so the model is free to learn to predict values in **any range**.

➤ Predicting House Prices – Building Model

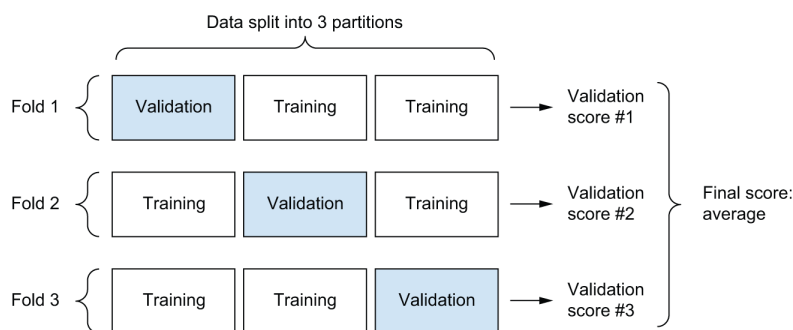
- We compile the model with the **mean_squared_error** loss function.
- **Mean Squared Error**
 - The **square of the difference** between the predictions and the targets.
 - This is a widely used loss function for **regression problems**.
- We monitor **Mean Absolute Error (MAE)** during training.
 - It is the **absolute value of the difference** between the predictions and the targets.
 - An **MAE** of 0.5 on this problem would mean our predictions are off by **\$50,000 on average**.
 - Remember the **target scaling** of factor 100,000.

➤ Validating – K-fold Validation

- We **split** the data into a **training** set and a **validation** set
 - To **evaluate** our model while we keep **adjusting** its parameters, such as the number of epochs used for training.
- With few data points, the validation set would end up being very small
 - The validation scores might change a lot depending on which data points we chose to use for validation and which we chose for training.
 - The **validation scores** might have a **high variance** with regard to the validation split.
 - This **prevent** us from **reliably evaluating** our model.

➤ Validating – K-fold Validation

- The best practice is to use **K-fold cross-validation**:
 - **Splitting** the available data into **K partitions** (typically $K = 4$ or 5)
 - **Instantiating K identical models**.
 - **Training** each one on **$K - 1$ partitions** while evaluating on the **remaining partition**.
- The validation score for the model is the **average of the K validation scores** obtained.



➤ Validating – K-fold Validation

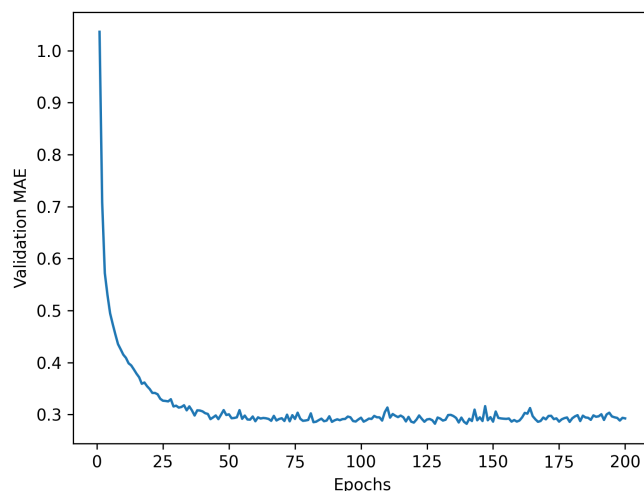
- If we use **4-fold** validation with **50 epochs**, we get the following result:

```
>>> [round(value, 3) for value in all_scores]
[0.298, 0.349, 0.232, 0.305]
>>> round(np.mean(all_scores), 3)
0.296
```

- The different runs show meaningfully **different validation** scores, from 0.232 to 0.349.
- The average (0.296) is a much **more reliable** metric than any single score.
- We are off by **\$29,600 on average**, which is significant considering that the prices range from \$60,000 to \$500,000.

➤ Predicting House Prices – Evaluating Model

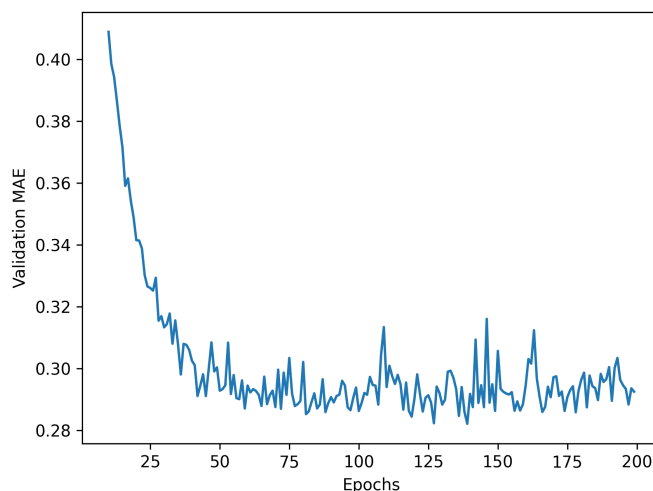
- We try training the model a bit longer for 200 epochs.
- We save the per-epoch validation score.
- We can compute the average of the per-epoch mean absolute error (MAE) scores for all folds and plot it.



Validation MAE by epoch.

➤ Predicting House Prices – Evaluating Model

- The validation MAE for the first few epochs is dramatically higher than the other values.
 - It is difficult to read the plot due to a scaling issue.
- We can omit the first 10 data points, which are on a different scale than the rest of the curve.
- According to this plot, validation MAE stops improving significantly after 120–140 epochs.
 - Past this point, we start overfitting.
- We can train a final production model on all of the training data, with the best parameters, and then look at its performance on the test data.



Validation MAE by epoch, excluding the first 10 data points.

➤ Predicting House Prices – Prediction

- With our scalar regression model calling `predict()` returns the model's **guess** for the **sample's price** in hundreds of thousands of dollars:

```
>>> predictions = model.predict(x_test)
>>> predictions[0]
array([2.834494], dtype=float32)
```

- The first district in the test set is predicted to have an **average home price** of about **\$283,000**.

Summary

- We looked at Multiclass Classification and started with an example to classify newswires using Reuters dataset.
- We learned about the specialized terms used in Regression.
- We looked at Regression by solving an example to predict the average home price in different districts in California.



References

- The content and figures used in these slides are extracted from the textbook **Deep Learning with Python, Third Edition - François Chollet.**

Next Session: Generalization



Any Questions?